

Contents

1	OBSERVER 2.0 — Docset v1.2.5 (Markdown)	1
1.1	000_README_DOCSET.md	1
1.2	doc_id: 000_README_DOCSET docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	1
2	OBSERVER Document Set (v1.2.5)	1
2.1	Scopo del docset	1
2.2	Contenuti canonici	1
2.2.1	Canonici (authoritative)	1
2.2.2	Supporto (verificabilità)	1
2.2.3	Specifiche tecniche	1
2.2.4	Use-cases (supporto, scenario-driven)	1
2.2.5	PDF & sorgente LaTeX	1
2.3	CONTRATTI DI GOVERNANCE (hard)	2
2.4	Regole operative (GUARDIAN - direct mode)	2
2.5	Support documents	2
2.6	001_PROJECT_OVERVIEW.md	2
2.7	doc_id: 001_PROJECT_OVERVIEW docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	2
3	OBSERVER 2.0 — Project Overview (allineato al codice)	2
3.1	1. In una frase	2
3.2	2. Il problema che risolve (retail avanzato)	3
3.3	3. Stato attuale (as-built): cosa è già operativo	3
3.3.1	3.1 Decision & research engine (maturo)	3
3.3.2	3.2 Operatività paper-first (baseline presente)	3
3.4	4. Aree di completamento v1.2.5 (obiettivo: scenario 1 paper robusto)	3
3.5	5. Architettura ad alto livello (layer)	3
3.6	6. Workflow operativo (tipico)	3
3.7	7. Nota su performance e promesse	3
3.8	002_PDR_OBSERVER.md	3
3.9	doc_id: 002_PDR_OBSERVER docset_version: 1.2.5 status: canonical last_updated: 2026-01-30	4
4	PDR — OBSERVER 2.0 (v1.2.5)	4
4.1	1. Executive summary	4
4.2	2. Target user e scenari	4
4.2.1	2.1 Retail avanzato disciplinato	4
4.2.2	2.2 Builder / quant retail	4
4.2.3	2.3 Scenari applicativi (v1.2.5)	4
4.3	3. Scope	4
4.3.1	3.1 In-scope (v1.2.5)	4
4.3.2	3.2 Out-of-scope (perimetro <i>non</i> incluso come requisito chiuso in v1.2.5)	4
4.4	4. Requisiti funzionali (FR)	5
4.4.1	FR-01 Offline-by-default	5
4.4.2	FR-02 Audit run tracciabile	5
4.4.3	FR-03 No future leak (conservativo)	5
4.4.4	FR-04 Determinismo e stabilità ranking	5
4.4.5	FR-05 Risk Engine (pre-trade e post-trade) — v1.2.5	5
4.4.6	FR-06 OMS + Execution log (paper-first) — v1.2.5	5
4.4.7	FR-07 Behavioral guardrails (“discipline mode”) — v1.2.5	5
4.4.8	FR-08 Monitoring & alerting — v1.2.5	5
4.4.9	FR-09 PHASE1 DataOps — prezzi: ingestion, halt governance, DQ (halt-aware)	6
4.5	5. Requisiti non funzionali (NFR)	6
4.6	6. Definition of Done (DoD) v1.2.5	6
4.7	7. Piano implementativo (esaustivo e strutturato)	6
4.7.1	Milestone M0 — Baseline certificabile (già esistente)	6
4.7.2	Milestone M1 — “Operational Safety” per Scenario 1 (paper robusto)	6
4.7.3	Milestone M2 — Alpha expansion controllata (Scenario 2)	6
4.7.4	Milestone M3 — Event-driven (Scenario 3) — solo dopo dati affidabili	7
4.7.5	Milestone M4 — Macro/Rotation e strategie avanzate (Scenari 4–5)	7

4.8	8. Rischi e mitigazioni (realistiche)	7
4.9	003_PDD_OBSERVER.md	7
4.10	doc_id: 003_PDD_OBSERVER docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	7
5	PDD — OBSERVER 2.0 (Design Document) v1.2.5	7
5.1	1. Design intent	7
5.2	2. Architettura a layer	7
5.2.1	2.1 Entry points (CLI / ops)	7
5.2.2	2.2 Application layer (src/)	7
5.2.3	2.3 Presentation layer (UI)	8
5.3	3. Dataflow end-to-end	8
5.4	4. Contratti di progettazione (policy)	8
5.4.1	4.1 Offline-by-default	8
5.4.2	4.2 No future leak	8
5.4.3	4.3 Auditability	8
5.5	5. Data layer (DuckDB)	8
5.6	6. Risk & Execution: stato e target v1.2.5	8
5.6.1	6.1 Stato as-built (baseline)	8
5.6.2	6.2 Target architetturale (v1.2.5)	8
5.7	7. Monitoring & alerting (v1.2.5)	9
5.8	8. Scenario-driven extensions	9
5.9	004_DDT_DATADictionary.md	9
5.10	doc_id: 004_DDT_DATADictionary docset_version: 1.2.5 status: canonical last_updated: 2026-01-30	9
6	DATADictionary (DuckDB) — v1.2.5	9
6.1	Panoramica	9
6.1.1	Convenzioni generali	9
6.1.2	Relazioni logiche (alto livello)	9
6.2	metadata	9
6.2.1	Colonne	10
6.2.2	Vincoli (DuckDB)	10
6.3	prices	10
6.3.1	Colonne	10
6.3.2	Vincoli (DuckDB)	10
6.4	dividends	10
6.4.1	Colonne	11
6.4.2	Vincoli (DuckDB)	11
6.5	recs	11
6.5.1	Colonne	11
6.5.2	Vincoli (DuckDB)	11
6.6	momentum_rankings	11
6.6.1	Colonne	12
6.6.2	Vincoli (DuckDB)	12
6.7	universes	12
6.7.1	Colonne	12
6.7.2	Vincoli (DuckDB)	12
6.8	universe_membership	12
6.8.1	Colonne	12
6.8.2	Vincoli (DuckDB)	12
6.9	ticker_mappings	12
6.9.1	Colonne	13
6.9.2	Vincoli (DuckDB)	13
6.10	ticker_halts	13
6.10.1	Colonne	13
6.10.2	Vincoli (DuckDB)	13
6.11	market_halts	13
6.11.1	Colonne	13
6.11.2	Vincoli (DuckDB)	13
6.12	sentiment_cache	14
6.12.1	Colonne	14

6.12.2	Vincoli (DuckDB)	14
6.13	data_gaps	14
6.13.1	Colonne	14
6.13.2	Vincoli (DuckDB)	14
6.14	dq_runs	15
6.14.1	Colonne	15
6.14.2	Vincoli (DuckDB)	15
6.15	dq_findings	15
6.15.1	Colonne	15
6.15.2	Vincoli (DuckDB)	15
6.16	dq_metrics_daily	16
6.16.1	Colonne	16
6.16.2	Vincoli (DuckDB)	16
6.17	audit_runs	16
6.17.1	Colonne	16
6.17.2	Vincoli (DuckDB)	16
6.18	audit_signal_decisions	16
6.18.1	Colonne	17
6.18.2	Vincoli (DuckDB)	17
6.19	execution_orders	17
6.19.1	Colonne	17
6.19.2	Vincoli (DuckDB)	17
6.20	execution_fills	18
6.20.1	Colonne	18
6.20.2	Vincoli (DuckDB)	18
6.21	audit_trades	18
6.21.1	Colonne	19
6.21.2	Vincoli (DuckDB)	20
6.22	audit_equity	20
6.22.1	Colonne	20
6.22.2	Vincoli (DuckDB)	20
6.23	005_TRACEABILITY_MATRIX.md	20
7	Traceability Matrix (Repo -> Docset) — v1.2.5	20
7.1	Scopo	20
7.2	0) Document set governance	20
7.3	A) Entrypoint e orchestrazione	21
7.4	B) Runner e utility operative (scripts/)	21
7.5	B2) PHASE1 DataOps (stabilità dati prezzi)	22
7.6	C) Data layer (DuckDB + migrazioni)	22
7.7	D) Execution + Risk + Monitoring	23
7.8	E) Core engine (src/core)	23
7.9	F) Forecast	23
7.10	G) NEWS-ALPHA (src/news_alpha)	24
7.11	H) Moduli applicativi (src/*.py)	24
7.12	I) UI Streamlit (pages/)	25
7.13	J) GUARDIAN tooling (operativo)	25
7.14	K) Tooling verifiche (src/tools)	26
7.15	Regola di accettazione (operativa)	26
7.16	006_REPO_BOM.md	26
7.17	doc_id: 006_REPO_BOM docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	26
8	Repository BOM (Bill of Materials) — v1.2.5	26
8.1	1. Scopo	26
8.2	2. Struttura ad alto livello	26
8.3	3. Runner e utility (scripts/)	27
8.4	4. UI Streamlit (pages/)	27
8.5	5. Package applicativi (src/)	27
8.6	6. Note di release “pulita”	27
8.7	007_PARAMETER_SNAPSHOT.md	27

8.8	doc_id: 007_PARAMETER_SNAPSHOT docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	28
9	Parameter Snapshot (estratto dal codice) — v1.2.5	28
9.1	1. Cost model (retail realism)	28
9.2	2. Tax model (Italia, semplificato)	28
9.3	3. Risk gate (baseline)	28
9.4	4. Forecast ranking (Wave 6)	28
9.5	5. NEWS-ALPHA deterministic sentiment	28
9.6	6. Execution (paper)	28
9.7	008_EVIDENCE_PACK.md	28
9.8	doc_id: 008_EVIDENCE_PACK docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	28
10	Evidence Pack — v1.2.5	28
10.1	1. Sanity checks	29
10.2	2. Data layer (DuckDB)	29
10.3	3. Pipeline sentinel (audit/backtest)	29
10.4	4. NEWS-ALPHA	29
10.5	5. Forecast ranking (Wave 6)	29
10.6	6. Execution (paper-first)	29
10.7	7. Packaging sessione	29
10.8	8. Docset governance	29
10.9	009_GAP_REGISTER.md	30
10.10	doc_id: 009_GAP_REGISTER docset_version: 1.2.5 status: canonical last_updated: 2026-01-26	30
10.11	Indice gap (ID stabili)	30
10.12	Dettagli gap (per ID)	30
10.12.1	GAP-OMS — Order Management System (OMS)	30
10.12.2	GAP-BROKER-ADAPTER — Live broker adapter + kill-switch	30
10.12.3	GAP-EXEC-LOG — Stable execution log	30
10.12.4	GAP-RISK-HARDEN — Hardened Risk Engine	30
10.12.5	GAP-GUARDRAILS — Behavioral guardrails	30
10.12.6	GAP-ALERTING — Alerting & notifications	31
10.12.7	GAP-DRIFT — Drift detection + model monitoring	31
10.12.8	GAP-CA-COLLECTOR — Corporate actions collector	31
10.12.9	GAP-MACRO-LAYER — Macro data layer + regime detection	31
10.12.10	GAP-PAIRS-ENGINE — Pairs trading engine	31
11	GAP Register — v1.2.5 (as-built vs target)	31
11.1	1. Scopo	31
11.2	2. Priorità e codifica	31
11.3	3. Gap principali per layer	32
11.3.1	Layer 1 — Risk & Execution (P0/P1)	32
11.3.2	Layer 2 — Alpha enhancement (P2)	32
11.3.3	Layer 3 — Feedback & Monitoring (P1)	32
11.3.4	Layer 4 — UI/UX operativa (P2)	32
11.4	4. Note di realismo	32
12	Technical Specs (docs/specs)	33
12.1	specs/AUDIT_LIFECYCLE_SPEC.md	33
13	AUDIT_LIFECYCLE_SPEC.md	33
13.1	Scopo	33
13.2	Stati (concettuali)	33
13.2.1	Stati pre-trade (eligibility)	33
13.2.2	Stati trade (execution)	33
13.3	Decision ledger	33
13.4	Regole di disclosure	33
13.5	specs/NEWS_ALPHA_SPEC.md	33
14	NEWS_ALPHA_SPEC.md	33
14.1	Scopo	33
14.2	Postura online (guardrail)	34

14.3 Pipeline RSS (collect)	34
14.4 Pipeline run (fixtures -> DuckDB)	34
14.5 History lane (GDELT)	34
14.6 specs/SENTINEL_RUNNER_SPEC.md	35
15 SENTINEL_RUNNER_SPEC.md	35
15.1 Scopo	35
15.2 Comandi supportati	35
15.3 Run ID	35
15.4 Transcript	35
15.5 Offline/Online posture	35
15.6 Disclosure defaults (retail)	35
15.7 specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md	36
16 WAVE6_FORECAST_STARS_RANKING_SPEC.md	36
16.1 Scopo	36
16.2 Input	36
16.3 Output	36
16.4 Algoritmo (passi)	36
16.4.1 Step 0 - Determine as-of date	36
16.4.2 Step 1 - Load candidate signals	36
16.4.3 Step 2 - Load calibration stats (audit_trades)	36
16.4.4 Step 3 - Shrinkage (robustezza piccoli campioni)	37
16.4.5 Step 4 - Sentiment adjustment	37
16.4.6 Step 5 - Forecast e confidence	37
16.4.7 Step 6 - Ranking e stars	37
16.5 Disclosure / retail notes	37

1 OBSERVER 2.0 — Docset v1.2.5 (Markdown)

Auto-assembled from the canonical documents in docs/.

1.1 000_README_DOCSET.md

**1.2 doc_id: 000_README_DOCSET docset_version: 1.2.5 status: canonical
last_updated: 2026-01-26**

2 OBSERVER Document Set (v1.2.5)

Build date: 2026-01-26

2.1 Scopo del docset

Questo docset definisce e governa **OBSERVER 2.0** in modalità *auditability-first* e *offline-by-default*.

- docs/ è la **source of truth**: i file Markdown canonici sono versionabili, diffabili e tracciabili.
- Il PDF docs/OBSERVER_v1.2.5.pdf è un **artefatto derivato** (render/packaging), non la sorgente primaria.
- La cartella .doc/ contiene la **libreria canonica derivata** (brief PROJ/TECH/DDT) generata via GUARDIAN.

2.2 Contenuti canonici

2.2.1 Canonici (authoritative)

- 001_PROJECT_OVERVIEW.md
- 002_PDR_OBSERVER.md
- 003_PDD_OBSERVER.md
- 004_DDT_DATADictionary.md

2.2.2 Supporto (verificabilità)

- 005_TRACEABILITY_MATRIX.md
- 006_REPO_BOM.md
- 007_PARAMETER_SNAPSHOT.md
- 008_EVIDENCE_PACK.md
- 009_GAP_REGISTER.md

2.2.3 Specifiche tecniche

- docs/specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md
- docs/specs/SENTINEL_RUNNER_SPEC.md
- docs/specs/NEWS_ALPHA_SPEC.md
- docs/specs/AUDIT_LIFECYCLE_SPEC.md

2.2.4 Use-cases (supporto, scenario-driven)

- docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md

2.2.5 PDF & sorgente LaTeX

- docs/OBSERVER_v1.2.5.pdf
- docs/LATEX_zip/OBSERVER_LATEX_CLEAN_v1.2.5_PROJECT.zip

2.3 CONTRATTI DI GOVERNANCE (hard)

Queste regole sono **contratti**: se vengono violate, aumenta il rischio di drift e incoerenza.

1. Canonici solo in docs/

- I documenti canonici (*authoritative*) devono vivere **solo** in docs/.
- La cartella .doc/ è riservata ai **derivati GUARDIAN** (brief/manifest/ops), non è un secondo set di canonici.

2. MkDocs è una VISTA derivata (mai source-of-truth)

- mkdocs/ contiene solo: configurazione, guide UI, API docs, e **stub** che includono i canonici da docs/.
- È vietato mantenere copie “vive” dei canonici dentro mkdocs/docs/ (es. 009_*.md, 010_*.md, ecc.).

3. Artefatti duplicabili in MkDocs solo se NON canonici

- Esempi: PDF, immagini, export HTML. Se copiati in mkdocs/docs/, devono essere considerati **artefatti derivati** e sincronizzati via script.

4. Post-TODO update (derivato)

- Dopo una patch/commit derivata da TODO (anche via prompt), si rigenerano solo i derivati:
 - GUARDIAN (sync/lint/derive), master MD, MkDocs build.
- I canonici si aggiornano **solo** quando cambia realmente la specifica/piano, non “per riflesso”.

5. Verificabilità

- Se viene introdotta una mappa (gap↔todo↔moduli), deve essere **derivata** (generata) o single-source-of-truth nel TODO, mai duplicata manualmente in più file.

2.4 Regole operative (GUARDIAN - direct mode)

Comandi standard (PowerShell/Windows):

- Stato docset: pyscripts/guardian.pystatus
- Allinea libreria canonica: pyscripts/guardian.pysync--clean
- Lint strutturale: pyscripts/guardian.pyLint
- Deriva i brief: pyscripts/guardian.pyderive

Regola d’oro: se cambia il codice, si fa prima **as-built inventory + delta**, poi si aggiornano i canonici e infine si rigenera .doc//manifest/checksum/PDF.

2.5 Support documents

- docs/010_MODULE_REGISTRY.md — module registry (as-built + target)
- docs/011_GAP_DERIVATION_MATRIX.md — derived-gap matrix

2.6 001_PROJECT_OVERVIEW.md

2.7 doc_id: 001_PROJECT_OVERVIEW docset_version: 1.2.5 status: canonical last_updated: 2026-01-26

3 OBSERVER 2.0 — Project Overview (allineato al codice)

Build date: 2026-01-26

3.1 1. In una frase

OBSERVER è un sistema *offline-by-default* che raccoglie segnali (news/analyst), li sottopone a gate di qualità/provenance, esegue audit/backtest conservativi (no future leak) e produce **ranking deterministici** (stars + confidence) con evidenze riproducibili.

3.2 2. Il problema che risolve (retail avanzato)

- Backtest non auditabile (leakage, universi non controllati, risultati non riproducibili).
- Dati “sporchi” (ticker mapping, prezzi mancanti, sospensioni, corporate actions).
- Segnali non comparabili (fonti diverse senza calibrazione/confidence).
- Operatività opaca (mancanza di transcript, run_id, fingerprint del codice).

3.3 3. Stato attuale (as-built): cosa è già operativo

3.3.1 3.1 Decision & research engine (maturo)

- **Audit engine** con regole conservative e ledger (audit_runs, audit_trades, audit_equity).
- **NEWS-ALPHA** deterministico (ingest/score/dedup) con output riproducibile.
- **Forecast ranking (Wave 6)**: shrinkage + confidence + percentile-to-stars (src/forecast/ranking.py).
- **Cost model** e **tax model IT** per realismo retail (src/core/*).
- **UI Streamlit** per monitoraggio e ispezione (app.py, pages/*).

3.3.2 3.2 Operatività paper-first (baseline presente)

- **Paper execution baseline**: genera ordini e scrive execution_orders / execution_fills (src/execution/paper_broker.py, scripts/execute.py).
- **Risk gate baseline**: sizing statico + cash reserve + max positions (src/risk/risk_engine.py).

Questa baseline è sufficiente per “paper trading dimostrativo”, ma non ancora per paper **robusto** né per live.

3.4 4. Aree di completamento v1.2.5 (obiettivo: scenario 1 paper robusto)

- 1) **RiskEngine evoluto (pre+post trade)**: stop/trailing (es. ATR-based), kill-switch, cooldown, limiti concentrazione/liquidità.
- 2) **OMS minimale**: lifecycle ordine/fill + posizioni/PNL accounting + reconciliation/idempotenza.
- 3) **Execution realism**: slippage/spread model + TCA report (evitare paper troppo ottimista).
- 4) **Monitoring & alerting**: anomaly (data gaps/execution) + model health (rolling metrics).
- 5) **Guardrails retail**: “discipline mode” con hard blocks e override logging.

Scenari applicativi e valutazione realistica: docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md.

3.5 5. Architettura ad alto livello (layer)

Layer	Componenti principali	Output
Data layer	DuckDB + migrations (src/db/*)	tabelle versionate + audit store
Signal layer	src/news_alpha/*, recs, mapping	segnali normalizzati e tracciati
Audit/Backtest	src/core/*	trades/equity + report
Forecast/Ranking	src/forecast/ranking.py	ranking deterministico + stars
Risk/Execution	src/risk/*, src/execution/*	ordini ammessi/negati + orders/fills
Monitoring/UI	src/monitoring/*, app.py, pages/*	dashboard, alert, drill-down

3.6 6. Workflow operativo (tipico)

- 1) **Setup / migrate**: crea/aggiorna schema DuckDB.
- 2) **Run audit**: ingest segnali → gates → backtest conservativo → audit tables + report.
- 3) **Forecast**: calibrazione su storico → stars/ranking deterministici.
- 4) **Execute (paper)**: dal ranking → risk gate → orders/fills persistiti.
- 5) **Monitor**: KPI operativi (data gaps, execution anomalies) + health modello.

3.7 7. Nota su performance e promesse

Qualunque stima di alpha/sharpe è una **ipotesi di ricerca** e va trattata come tale: validazione out-of-sample, costi/slippage conservativi e monitoring sono parte integrante del prodotto.

3.8 002_PDR_OBSERVER.md

3.9 doc_id: 002_PDR_OBSERVER docset_version: 1.2.5 status: canonical
last_updated: 2026-01-30

4 PDR — OBSERVER 2.0 (v1.2.5)

Build date: 2026-01-30

4.1 1. Executive summary

OBSERVER è una piattaforma “retail advanced” per:

- generare segnali (NEWS-ALPHA / analyst recs)
- trasformarli in ranking deterministici (stars + confidence)
- produrre audit trail e verificabilità (no future leak, provenance, run_id, fingerprint)
- abilitare progressivamente operatività **paper-first** e, in estensione, **live** con broker adapter
- garantire **DataOps** ripetibili (ingest, halt governance, DQ) per stabilità operativa e certification-grade runs

Il requisito primario è la **governance operativa**: ogni azione deve essere auditabile, riproducibile e “safe-by-design”.

4.2 2. Target user e scenari

4.2.1 2.1 Retail avanzato disciplinato

Vuole un workflow giornaliero/settimanale ripetibile, controlli rischio, e trasparenza (perché buy/skip?).

4.2.2 2.2 Builder / quant retail

Vuole estendere universi/dati/modelli, ma con vincoli di audit e regressione (test, schema versionato, lint).

4.2.3 2.3 Scenari applicativi (v1.2.5)

Gli scenari operativi sono formalizzati in:

- docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md

Policy di roadmap: prima si chiude il Layer “Operational Safety” (Risk/OMS/Execution/Monitoring), poi si scala su scenari alpha aggiuntivi.

4.3 3. Scope

4.3.1 3.1 In-scope (v1.2.5)

- Data layer DuckDB (schema versionato + audit tables)
- **PHASE1 DataOps (stabilità dati prezzi):**
 - seed closures → market_halts
 - overlay halts YAML → market_halts / ticker_halts
 - ingest incrementale prezzi (PriceBackfiller)
 - DQ prezzi **halt-aware** con persistenza (dq_runs, dq_findings, dq_metrics_daily)
 - Streamlit Control Room DataOps (editing halts.yml + run buttons + viste tabelle)
- Runner CLI: sentinel/news_alpha/forecast/execute (paper), pack_session, ops workflow
- Risk gate baseline e paper execution baseline (già presenti) da evolvere
- Monitoring base + UI Streamlit multi-pagina

4.3.2 3.2 Out-of-scope (perimetro *non* incluso come requisito chiuso in v1.2.5)

- Broker live production-ready (richiede adapter specifici e test con broker)
- Intraday / microstruttura / HFT
- Dataset proprietari/licenziati non inclusi
- Consulenza finanziaria o fiscale (solo educational tooling + disclaimer)

4.4 4. Requisiti funzionali (FR)

4.4.1 FR-01 Offline-by-default

Nessuna chiamata rete senza --online o equivalente.

Acceptance: runner in assenza di flag opera senza rete e documenta la modalità.

4.4.2 FR-02 Audit run tracciabile

Ogni run deve avere run_id, transcript, e code_fingerprint.

Acceptance: artefatti in reports/ e record in audit_runs.

4.4.3 FR-03 No future leak (conservativo)

Le regole T+1 e la logica di enterability devono essere verificabili e registrate.

Acceptance: per segnali data D, entry non può essere ≤ D; reason_code esplicito.

4.4.4 FR-04 Determinismo e stabilità ranking

A parità di dati e codice, ranking identico (tie-break deterministici).

Acceptance: sort stabile; output JSON con ordering invariato.

4.4.5 FR-05 Risk Engine (pre-trade e post-trade) — v1.2.5

Il sistema deve implementare un **modulo esplicito** di risk management separato dal ranking.

Stato as-built: presente gate base (cash reserve + max_positions + sizing statico).

Target v1.2.5 (minimo operativo):

- pre-trade: sizing (risk-aware), limiti concentrazione, liquidity/turnover caps
- post-trade: stop-loss/trailing (ATR-based o equivalente), cooldown, kill-switch
- output: decision list con reason_code e parametri applicati

Acceptance (v1.2.5):

- esiste una funzione/entrypoint che, dato ranking + portafoglio, produce ordini ammessi/negati
- stop-loss genera exit orders con audit trail
- kill-switch blocca nuove aperture e forza riduzione rischio secondo policy

4.4.6 FR-06 OMS + Execution log (paper-first) — v1.2.5

Il sistema deve registrare l'intero lifecycle ordine/fill e lo stato posizione.

Stato as-built: paper execution scrive execution_orders/execution_fills.

Target v1.2.5:

- OMS minimale: order states (CREATED/SUBMITTED/FILLED/REJECTED/CANCELLED)
- posizione/PNL accounting (cash, holdings, realized/unrealized)
- reconciliation e idempotenza (no double fills, run_id consistent)

Acceptance (v1.2.5):

- scripts/execute.py genera ordini e aggiorna posizioni
- esiste report di execution (fills, fees, slippage model) e ledger aggiornato

4.4.7 FR-07 Behavioral guardrails (“discipline mode”) — v1.2.5

Il sistema deve prevenire misuse retail (override non controllati).

Acceptance:

- regole minime: cooling-off, max loss day/week, override logging con motivazione
- modalità “hard blocks” configurabile

4.4.8 FR-08 Monitoring & alerting — v1.2.5

Monitoraggio operativo e di modello (drift) con allarmi.

Acceptance:

- anomaly alerts: execution anomalies, data gaps, schema mismatch
- model health: rolling metrics (hit-rate, IC proxy, forecast vs realized)
- alert center (UI o log) con soglie configurabili

4.4.9 FR-09 PHASE1 DataOps — prezzi: ingestion, halt governance, DQ (halt-aware)

Il sistema deve offrire un workflow ripetibile per:

- governare le chiusure/halts di mercato e ticker (seed + overlay) con provenienza
- ingest incrementale prezzi (con data_gaps) senza corrompere il DB
- eseguire controlli DQ su calendario business-day **escludendo** gli intervalli in market_halts e ticker_halts
- persistere i risultati DQ in tabelle dedicate (dq_runs, dq_findings, dq_metrics_daily) in modo idempotente

Acceptance (PHASE1):

- CLI DataOps disponibili come moduli -msrc.tools.* + pagina Streamlit pages/11_DataOps_Control_Room.py
- un run DQ produce record coerenti e ripetibili (run_id idempotente) in dq_*

4.5 5. Requisiti non funzionali (NFR)

- Riproducibilità (run deterministiche salvo timestamp)
- Robustezza (data gaps non corrompono DB; circuit breaker)
- Trasparenza (reason_code per ogni decisione)
- Sicurezza (segreti via env; no hardcoding)
- Portabilità (Windows/PowerShell supportata: py)

4.6 6. Definition of Done (DoD) v1.2.5

- 005_TRACEABILITY_MATRIX.md copre 100% componenti rilevanti
- guardianlint senza errori
- pytest passa nello snapshot (inclusi test minimi PHASE1 DataOps)
- sentinel/news_alpha/forecast/execute eseguono su DB presente
- evidenze in 008_EVIDENCE_PACK.md eseguibili e coerenti

4.7 7. Piano implementativo (esaustivo e strutturato)

4.7.1 Milestone M0 — Baseline certificabile (già esistente)

- audit/backtest/ranking deterministici, offline-by-default, UI base

4.7.2 Milestone M1 — “Operational Safety” per Scenario 1 (paper robusto)

Obiettivo: scenario 1 eseguibile in paper con controlli rischio e audit completi.

Deliverable:

- RiskEngine v1 (pre+post trade: stop/trailing/cooldown/kill-switch)
- OMS minimale con posizione/PNL accounting
- Paper execution con slippage model base + TCA report
- Guardrails minimi (discipline mode)
- Monitoring base + alerting (execution/data gaps)

Acceptance:

- un utente può selezionare ranking e vedere ordini simulati, controllati e registrati
- stop-loss genera exit con audit trail
- session pack deterministico con report execution

4.7.3 Milestone M2 — Alpha expansion controllata (Scenario 2)

Deliverable:

- factor library estesa (minimo: value/quality con definizioni e governance)
- dynamic weighting (regime-aware “base”)
- attribution “minimo” per fattore e per contributo a return

Acceptance:

- ranking multi-factor riproducibile + report attribution

4.7.4 Milestone M3 — Event-driven (Scenario 3) — solo dopo dati affidabili

Deliverable:

- corporate actions collector + event engine (conservative timing)
- risk limits event-driven

4.7.5 Milestone M4 — Macro/Rotation e strategie avanzate (Scenari 4–5)

Solo dopo stabilizzazione execution/risk/monitoring.

4.8 8. Rischi e mitigazioni (realistiche)

- Paper troppo ottimista → slippage/spread model early + stress test
- Data gaps live → circuit breaker + procedure override tracciata
- Alpha decay → OOS rigoroso + monitoring; nessuna promessa di performance
- Regulatory/communication → disclaimer chiaro + educational stance

4.9 003_PDD_OBSERVER.md

4.10 doc_id: 003_PDD_OBSERVER docset_version: 1.2.5 status: canonical
last_updated: 2026-01-26

5 PDD — OBSERVER 2.0 (Design Document) v1.2.5

Build date: 2026-01-26

5.1 1. Design intent

OBSERVER adotta un'impostazione **auditability-first**:

- determinismo e riproducibilità (run_id, transcript, code_fingerprint)
- timing conservativo (no future leak, regola T+1)
- data governance (ticker mapping time-bounded, universe membership)
- output “retail usable” ma verificabile (stars + confidence + drill-down)
- evoluzione *paper-first* verso live, senza compromettere audit e safety.

5.2 2. Architettura a layer

5.2.1 2.1 Entry points (CLI / ops)

- scripts/sentinel.py: orchestrazione (migrate/test/run/certify/verify/forecast/status)
- scripts/news_alpha.py: NEWS-ALPHA runner (collect/run/status)
- scripts/execute.py: execution runner (paper-first)
- scripts/ops_run_session.py / scripts/pack_session.py: workflow e packaging deterministico
- scripts/guardian.py: governance documentale (direct mode)

5.2.2 2.2 Application layer (src/)

- src/db/*: schema owner + connection + audit store
- src/core/*: audit engine, cost/tax, ticker normalization
- src/news_alpha/*: ingest/scoring/dedup + persistence
- src/forecast/ranking.py: forecast/stars/ranking deterministici
- src/risk/risk_engine.py: risk gate (baseline) → evoluzione v1.2.5
- src/execution/paper_broker.py: paper broker baseline (orders/fills)
- src/monitoring/*: metriche e controlli operativi (baseline)

5.2.3 2.3 Presentation layer (UI)

- app.py + pages/*: console Streamlit (status, runs, ranking, monitoring).

5.3 3. Dataflow end-to-end

- 1) **Signals**: recs (analyst/news-derived) + eventuali staging tables
- 2) **Normalization & mapping**: ticker_mappings time-bounded + normalizzazione SQL
- 3) **Universe filter**: universe_membership (survivorship-bias control)
- 4) **Timing & feasibility**: join con prices e regola conservativa $\text{MIN}(\text{prices.date}) > \text{signal_date}$
- 5) **Audit/backtest**: produce audit_trades + audit_equity con costi/tasse
- 6) **Forecast**: calibrazione su audit_trades → ranking deterministico + stars
- 7) **Risk gate**: ranking → proposte ordini → decisioni ammessi/negati (reason_code)
- 8) **Execution (paper)**: scrive execution_orders / execution_fills
- 9) **Monitoring**: KPI e anomaly detection su dati/execution/modello
- 10) **UI**: drill-down e controllo operativo.

5.4 4. Contratti di progettazione (policy)

5.4.1 4.1 Offline-by-default

Qualunque accesso rete deve essere esplicito (flag/env). Il default è offline.

5.4.2 4.2 No future leak

- entry date strettamente successiva al segnale
- disclosure e reason_code per qualunque shift/skip.

5.4.3 4.3 Auditability

- run_id presente in ogni artefatto importante (DB e filesystem)
- code_fingerprint per correlare risultati a snapshot di codice.

5.5 5. Data layer (DuckDB)

- Owner canonico: src/db/migrate.py
- DB locale: data/sentinel_alpha.db
- Tabelle chiave: audit_*, recs, prices, universe_membership, ticker_mappings, execution_orders, execution_fills, sentiment_cache, data_gaps.

Riferimento: 004_DDT_DATADictionary.md.

5.6 6. Risk & Execution: stato e target v1.2.5

5.6.1 6.1 Stato as-built (baseline)

- risk gate: cash reserve, max positions, sizing statico (notional cap)
- paper broker: ordine “MARKET” simulato, fill immediato, fees via CostModel
- persistenza: execution_orders + execution_fills

5.6.2 6.2 Target architetturale (v1.2.5)

Obiettivo: passare da “paper dimostrativo” a **paper robusto**.

Componenti mancanti (design):

- **OMS minimal**: lifecycle ordine, idempotenza, reconciliation, position keeping
- **Post-trade risk**: stop/trailing, cooldown, kill-switch
- **Execution realism**: slippage/spread + partial fill simulation (anche se semplificata)
- **TCA**: confronto forecast vs realized e tracking costi effettivi

Interfacce consigliate:

- BrokerAdapter (paper/live) con contract uniforme (submit/cancel/poll)
- OrderIntent (snapshot del ranking + vincoli applicati) per audit e replay.

5.7 7. Monitoring & alerting (v1.2.5)

- **Operational**: data gaps, schema mismatch, execution anomalies
- **Model health**: rolling hit-rate / IC proxy, forecast vs realized, drift flags
- **Alerting**: log/DB flags + UI “alert center” (minimo)

5.8 8. Scenario-driven extensions

Gli scenari e i requisiti aggiuntivi sono descritti in:

- docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md

Politica: prima Layer 1 (safety), poi alpha expansion.

5.9 004_DDT_DATADictionary.md

5.10 doc_id: 004_DDT_DATADictionary docset_version: 1.2.5 status: canonical last_updated: 2026-01-30

6 DATADictionary (DuckDB) — v1.2.5

Build date: 2026-01-30

6.1 Panoramica

OBSERVER / SENTINEL-ALPHA utilizza **DuckDB** come single source of truth locale.

Il modello dati e’ progettato per:

- riproducibilita’ e auditability (run_id, code_fingerprint, transcript)
- backtest “certification-grade” (no survivorship bias, ticker mappings time-bounded, halts)

- realismo retail (costi transazione, tasse IT, dividendi come estensione)

6.1.1 Convenzioni generali

- DATE e' usato per la logica di sessione (daily).
- TIMESTAMPTZ/TIMESTAMP sono usati per logging e provenance. Quando possibile, usare UTC.
- Le chiavi sono implementate come PRIMARYKEY o UNIQUE (in alcuni casi DuckDB materializza la chiave come UNIQUE).

6.1.2 Relazioni logiche (alto livello)

- audit_runs 1..N audit_trades / audit_equity / audit_signal_decisions / data_gaps
 - dq_runs 1..N dq_findings / dq_metrics_daily
 - metadata 1..N prices / dividends
 - recs alimenta audit_signal_decisions e, indirettamente, audit_trades
-

6.2 metadata

Anagrafica strumenti: ticker canonico e attributi di mercato/settore, inclusi flag fiscali (Tobin/FTT) e simboli provider.

6.2.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico interno (chiave primaria).
sector	VARCHAR	YES	Settore economico (granularita' libera).
market	VARCHAR	YES	Etichetta mercato/regione (es. US, EU, ITALY).
currency	VARCHAR	YES	Valuta di quotazione (ISO-4217, es. USD, EUR).
is_tobin_tax	BOOLEAN	YES	Flag: applica Tobin/FTT (Italia) come costo transazione addizionale.
yf_symbol	VARCHAR	YES	Simbolo provider yfinance (se diverso dal ticker canonico).
stooq_symbol	VARCHAR	YES	Simbolo provider stooq (se diverso dal ticker canonico).
instrument_type	VARCHAR	YES	Classe strumento (EQUITY, ETF, DERIVATIVE, ...).
ftt_rate	DOUBLE	YES	Aliquota FTT come frazione del notional (es. 0.001 = 0.10%).

6.2.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker)

6.3 prices

Prezzi giornalieri (barre daily): close (obbligatorio) + open opzionale, con provenance e timestamp di inserimento.

6.3.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	YES	Data sessione (daily).
ticker	VARCHAR	YES	Ticker canonico.
price	FLOAT	YES	Prezzo di chiusura (close).
open_price	FLOAT	YES	Prezzo di apertura (open), se disponibile.
source	VARCHAR	YES	Provenienza riga (legacy/yfinance/stooq/...).
fetch_at	TIMESTAMP	YES	Timestamp inserimento/aggiornamento riga.

6.3.2 Vincoli (DuckDB)

- UNIQUE: UNIQUE(date, ticker)

6.4 dividends

Eventi dividendo per realismo retail (ex-date e pay-date). Nello snapshot lo schema e' presente; l'applicazione cashflow e' incrementale.

6.4.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico.
ex_date	DATE	NO	Data ex-dividend.
pay_date	DATE	YES	Data pagamento.
amount	DOUBLE	YES	Importo dividendo per share/unit.
currency	VARCHAR	YES	Valuta importo.
source	VARCHAR	YES	Provenienza dato.
fetch_at	TIMESTAMP	YES	Timestamp inserimento.

6.4.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker, ex_date)

6.5 recs

Store segnali/raccomandazioni (news/analyst): per data di pubblicazione, ticker, firm e rating, con sentiment score e metadati.

6.5.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	YES	Data pubblicazione segnale (legacy compatible).
ticker	VARCHAR	YES	Ticker del segnale (puo' essere non canonico: normalizzato/mappato a runtime).
firm	VARCHAR	YES	Fonte/firm (es. banca d'affari, provider news).
rating	VARCHAR	YES	Rating discreto (BUY/HOLD/DOWNGRADE o analogo).
sentiment_score	DOUBLE	YES	Sentiment in [-1,+1] (deterministico, cacheabile).
headline	VARCHAR	YES	Titolo/news headline associata al segnale.
source_url	VARCHAR	YES	URL sorgente (se disponibile).
universe_id	VARCHAR	YES	Universe associato al segnale (ALL/US/EU o custom).
published_at	TIMESTAMP	YES	Timestamp pubblicazione (se disponibile).

6.5.2 Vincoli (DuckDB)

- UNIQUE: UNIQUE(date, ticker, firm)

6.6 momentum_rankings

Ranking momentum giornaliero per ticker (feature/gate) con rendimento periodo e segnali discreti.

6.6.1 Colonne

Colonna	Tipo	NULL?	Semantica
date	DATE	NO	Data calcolo ranking.
ticker	VARCHAR	NO	Ticker canonico.
monthly_return	FLOAT	YES	
rank_pos	INTEGER	YES	
signal	VARCHAR	YES	Segnale discreto (es. LONG/SHORT/HOLD).

6.6.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(date, ticker)

6.7 universes

Catalogo universi (es. ALL/US/EU) per filtrare segnali e controllare survivorship bias.

6.7.1 Colonne

Colonna	Tipo	NULL?	Semantica
universe_id	VARCHAR	NO	ID universe (chiave primaria).
name	VARCHAR	YES	Nome descrittivo.
market	VARCHAR	YES	Mercato/regione prevalente (US/EU/MULTI...).
description	VARCHAR	YES	Descrizione testuale.

6.7.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(universe_id)

6.8 universe_membership

Membership storica (ticker in universo) con intervalli temporali: essenziale per backtest senza survivorship bias.

6.8.1 Colonne

Colonna	Tipo	NULL?	Semantica
universe_id	VARCHAR	NO	ID universe.
ticker	VARCHAR	NO	Ticker canonico.
start_date	DATE	NO	Inizio validita' membership (inclusivo).
end_date	DATE	YES	Fine validita' (inclusivo, NULL=open-ended).
source	VARCHAR	YES	Fonte dataset membership.
notes	VARCHAR	YES	Note/annotazioni.

6.8.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(universe_id, ticker, start_date)

6.9 ticker_mappings

Mappature alias->canonico time-bounded (corporate actions, cambio simbolo) per evitare mismatch ticker.

6.9.1 Colonne

Colonna	Tipo	NULL?	Semantica
alias_ticker	VARCHAR	NO	Simbolo alias (come appare nel dato grezzo).
canonical_ticker	VARCHAR	YES	Simbolo canonico interno.
start_date	DATE	NO	Inizio validita' mapping.
end_date	DATE	YES	Fine validita' mapping (NULL=open-ended).
source	VARCHAR	YES	Fonte mapping (manual, provider, corporate actions...).
notes	VARCHAR	YES	Note.

6.9.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(alias_ticker, start_date)

6.10 ticker_halts

Halt su singolo ticker: finestra temporale di non-tradabilita' e reason/provenance.

6.10.1 Colonne

Colonna	Tipo	NULL?	Semantica
ticker	VARCHAR	NO	Ticker canonico.
start_date	DATE	NO	Inizio halt.
end_date	DATE	YES	Fine halt (NULL=open-ended).
reason	VARCHAR	YES	Motivo halt (sospensione, delisting, market maker, ecc.).
source	VARCHAR	YES	Fonte informazione.

6.10.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(ticker, start_date)

6.11 market_halts

Halt di mercato/paese (es. circuit breaker, chiusure straordinarie) per execution feasibility.

6.11.1 Colonne

Colonna	Tipo	NULL?	Semantica
market	VARCHAR	NO	Mercato/regione (es. US, EU).
start_date	DATE	NO	Inizio halt.
end_date	DATE	YES	Fine halt.
reason	VARCHAR	YES	Motivo halt (circuit breaker, chiusure, ecc.).
source	VARCHAR	YES	Fonte informazione.

6.11.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(market, start_date)

6.12 sentiment_cache

Cache locale deterministica del sentiment (testo normalizzato + hash) per ripetibilit  e audit.

6.12.1 Colonne

Colonna	Tipo	NULL?	Semantica
text_hash	VARCHAR	NO	Hash del testo normalizzato (chiave primaria).
text	VARCHAR	YES	Testo normalizzato (troncato a 2000 char).
score	DOUBLE	YES	Sentiment score in [-1,+1].
model	VARCHAR	YES	Modello usato (vader/lexicon).
computed_at	TIMESTAMP	YES	Timestamp calcolo (UTC).

6.12.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(text_hash)

6.13 data_gaps

Log operativo di gap/backfill (prezzi/news): include intervalli richiesti, esito, righe inserite e reason_code standardizzato.

6.13.1 Colonne

Colonna	Tipo	NULL?	Semantica
kind	VARCHAR	YES	Tipo gap (prices/news/...).
ticker	VARCHAR	YES	Ticker target (se applicabile).
start_date	DATE	YES	Inizio finestra da ottenere (normalizzata).
end_date	DATE	YES	Fine finestra da ottenere.
requested_at	TIMESTAMP	YES	Timestamp richiesta.
status	VARCHAR	YES	SUCCESS/FAILED/SKIPPED.
provider	VARCHAR	YES	Provider (yfinance/stooq/gdelt/...).
message	VARCHAR	YES	Messaggio sintetico esito.
rows_inserted	INTEGER	YES	Numero righe inserite.
error	VARCHAR	YES	Errore raw (se presente).
duration_ms	INTEGER	YES	Durata operazione.
run_id	VARCHAR	YES	Run che ha richiesto il backfill (se presente).
rows_upserted	INTEGER	YES	Numero righe inserite + aggiornate.
reason_code	VARCHAR	YES	Reason code standardizzato (per KPI data quality).
requested_start_date	DATE	YES	Inizio richiesto originario (prima di clamp/parse).
requested_end_date	DATE	YES	Fine richiesta originaria.
obtained_start_date	DATE	YES	Min date effettivamente ottenuta.
obtained_end_date	DATE	YES	Max date effettivamente ottenuta.

6.13.2 Vincoli (DuckDB)

- (nessun vincolo dichiarato)

6.14 dq_runs

Registro esecuzioni Data Quality (PHASE1 DataOps): una riga per run DQ con finestra e stato.

6.14.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	Identificativo run DQ (chiave primaria).
kind	VARCHAR	YES	Tipo DQ (es. DQ_PRICES).
started_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp inizio (UTC consigliato).
finished_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp fine.
asof_date	DATE	YES	Data as-of del controllo (daily).
window_days	INTEGER	YES	Finestra lookback in giorni calendario.
status	VARCHAR	YES	SUCCESS/FAILED/SKIPPED.
notes	VARCHAR	YES	Note sintetiche (parametri + conteggi).
error	VARCHAR	YES	Errore raw se FAILED.

6.14.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id)

6.15 dq_findings

Dettaglio findings DQ (PHASE1): missing/stale/invalid, compressi per intervallo.

6.15.1 Colonne

Colonna	Tipo	NULL?	Semantica
finding_id	VARCHAR	NO	ID finding (chiave primaria).
run_id	VARCHAR	YES	Run DQ associato.
kind	VARCHAR	YES	Tipo finding (PRICE_MISSING/PRICE_STALE/...).
severity	VARCHAR	YES	Severità (INFO/WARN/ERROR).
market	VARCHAR	YES	Mercato derivato (US/EU/...).
ticker	VARCHAR	YES	Ticker canonico.
start_date	DATE	YES	Inizio intervallo finding.
end_date	DATE	YES	Fine intervallo finding.
count	INTEGER	YES	Numero giorni nel finding (best-effort).
message	VARCHAR	YES	Messaggio sintetico.
created_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp creazione record.

6.15.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(finding_id)

6.16 dq_metrics_daily

Metriche aggregate per run DQ (PHASE1) per mercato + metrica.

6.16.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	Run DQ.
asof_date	DATE	NO	Data as-of.
market	VARCHAR	NO	Mercato (ALL/US/EU/...).
metric	VARCHAR	NO	Nome metrica (tickers, findings, invalid_rows, duration_ms, ...).
value	DOUBLE	YES	Valore numerico.
created_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp inserimento.

6.16.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id, asof_date, market, metric)

6.17 audit_runs

Registro run di audit/certificazione: configurazione, universi, fingerprint del codice e stato esecuzione.

6.17.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	Identificativo run (chiave primaria, deterministico/UUID-like).
started_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp inizio run (UTC consigliato).
finished_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp fine run.
status	VARCHAR	YES	Stato: RUNNING/SUCCESS/FAILED.
universe_id	VARCHAR	YES	Universe testato.
holding_period_sessions	INTEGER	YES	Holding period in sessioni (T+N, default definito nella pipeline).
config_json	VARCHAR	YES	Configurazione run serializzata (string JSON).
code_fingerprint	VARCHAR	YES	Fingerprint del codice (hash) per audit.
notes	VARCHAR	YES	Note manuali / annotazioni.
error	VARCHAR	YES	Messaggio errore (se FAILED).

6.17.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id)

6.18 audit_signal_decisions

Decision ledger opzionale: traccia segnali eseguiti o scartati (dedup, halt, skip) e date ‘intese’ vs effettive.

6.18.1 Colonne

Colonna	Tipo	NULL?	Semantica
decision_id	VARCHAR	NO	ID decisione (chiave primaria).
run_id	VARCHAR	YES	ID run.
signal_date	DATE	YES	Data segnale.
ticker_original	VARCHAR	YES	Ticker originale.
ticker	VARCHAR	YES	Ticker canonico effettivo.
firm	VARCHAR	YES	Fonte segnale.
rating	VARCHAR	YES	Rating.
universe_id	VARCHAR	YES	Universe.
intended_buy_date	DATE	YES	Buy date intesa (prima di shift).
buy_date	DATE	YES	Buy date effettiva.
exec_shift_sessions	INTEGER	YES	Shift buy.
intended_sell_date	DATE	YES	Sell date intesa.
sell_date	DATE	YES	Sell date effettiva.
exit_shift_sessions	INTEGER	YES	Shift sell.
decision	VARCHAR	YES	EXECUTED / SKIPPED / DROPPED_DEDUP.
skip_reason	VARCHAR	YES	Motivo skip (missing price, halt, policy).
halt_reason	VARCHAR	YES	Motivo halt (ticker/market).
created_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp creazione record.

6.18.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(decision_id)

6.19 execution_orders

Ledger ordini di esecuzione (paper/real): entita' "ordine" con stato e parametri di submit.

6.19.1 Colonne

Colonna	Tipo	NULL?	Semantica
order_id	VARCHAR	NO	ID ordine (chiave primaria).
run_id	VARCHAR	YES	FK logica verso audit_runs.run_id (se presente).
created_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp creazione ordine (UTC consigliato).
ticker	VARCHAR	YES	Ticker canonico target.
side	VARCHAR	YES	Lato ordine: BUY/SELL.
quantity	DOUBLE	YES	Quantita' (shares/unit).
order_type	VARCHAR	YES	Tipo ordine (MARKET/LIMIT/...).
limit_price	DOUBLE	YES	Prezzo limite se LIMIT, altrimenti NULL.
status	VARCHAR	YES	Stato ordine (NEW/FILLED/CANCELLED/...).
notes	VARCHAR	YES	Note/testo libero.

6.19.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL

- PRIMARY KEY: PRIMARY KEY(order_id)

6.20 execution_fills

Ledger fills/eseguiti: ogni fill rappresenta un'esecuzione (anche parziale) di un ordine.

6.20.1 Colonne

Colonna	Tipo	NULL?	Semantica
fill_id	VARCHAR	NO	ID fill (chiave primaria).
order_id	VARCHAR	YES	FK logica verso execution_orders.order_id.
run_id	VARCHAR	YES	FK logica verso audit_runs.run_id (se presente).
filled_at	TIMESTAMP WITH TIME ZONE	YES	Timestamp fill (UTC consigliato).
ticker	VARCHAR	YES	Ticker canonico.
side	VARCHAR	YES	Lato eseguito: BUY/SELL.
quantity	DOUBLE	YES	Quantita' eseguita.
fill_price	DOUBLE	YES	Prezzo di esecuzione.
fees	DOUBLE	YES	Commissioni applicate (valuta di conto).
notes	VARCHAR	YES	Note/testo libero.

6.20.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(fill_id)

6.21 audit_trades

Trade ledger per run: ogni trade deriva da un segnale, include shift esecutivi, motivi di uscita e performance lorda/netta.

6.21.1 Colonne

Colonna	Tipo	NULL?	Semantica
trade_id	VARCHAR	NO	ID trade (chiave primaria).
run_id	VARCHAR	YES	FK logica verso audit_runs.run_id.
signal_date	DATE	YES	Data segnale originale (giorno di pubblicazione).
buy_date	DATE	YES	Data entrata effettiva (con shift e vincoli).
sell_date	DATE	YES	Data uscita effettiva.
exit_reason	VARCHAR	YES	Motivo uscita (TARGET/HOLDING_PERIOD/HALT/FALLBACK_LAST_PRICE...).
exit_is_fallback	BOOLEAN	YES	Flag: uscita fallback (prezzo last known).
ticker	VARCHAR	YES	Ticker effettivo canonico usato per prezzi e membership.
firm	VARCHAR	YES	Fonte segnale.
rating	VARCHAR	YES	Rating del segnale.
market	VARCHAR	YES	Mercato (derivato da metadata o regole).
sector	VARCHAR	YES	Settore (derivato da metadata).
mom_status	VARCHAR	YES	Esito gate momentum (se applicato).
risk_vol	DOUBLE	YES	Volatilità/proxy rischio usato nella gestione posizione (se presente).
is_tobin_tax	BOOLEAN	YES	Flag FTT applicata.
sentiment_score	DOUBLE	YES	Sentiment associato al segnale.
buy_price	DOUBLE	YES	Prezzo entrata (close o regola scelta).
sell_price	DOUBLE	YES	Prezzo uscita.
gross_return_pct	DOUBLE	YES	Rendimento lordo (%) pre-costi e pre-tasse.
cost_pct	DOUBLE	YES	Costo transazione totale (%) usato nel modello.
net_return_pct	DOUBLE	YES	Rendimento netto (%) dopo costi (tasse tracciate altrove).
trade_score	DOUBLE	YES	Score del trade (se applicato dal sistema).
universe_id	VARCHAR	YES	Universe in cui il trade e' stato eseguito/valutato.
ticker_original	VARCHAR	YES	Ticker originale del segnale (prima di normalize/mapping).
instrument_type	VARCHAR	YES	Classe strumento.
ftt_pct	DOUBLE	YES	FTT effettiva usata come percentuale del notional.
exec_shift_sessions	INTEGER	YES	Shift entrata (sessioni) per vincoli T+1 / prezzi mancanti / halt.
exit_shift_sessions	INTEGER	YES	Shift uscita (sessioni).
halt_reason	VARCHAR	YES	Motivo di halt (ticker/market) se ha impattato l'esecuzione.

6.21.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(trade_id)

6.22 audit_equity

Equity curve giornaliera per run: equity, cash, investito, numero posizioni e tasse pagate.

6.22.1 Colonne

Colonna	Tipo	NULL?	Semantica
run_id	VARCHAR	NO	ID run.
date	DATE	NO	Data (daily).
equity	DOUBLE	YES	Equity totale (cash + posizioni mark-to-market).
cash	DOUBLE	YES	Cassa disponibile.
invested	DOUBLE	YES	Notional investito (approssimazione).
positions	INTEGER	YES	Numero posizioni aperte.
tax_paid	DOUBLE	YES	Tasse pagate cumulative o per giorno (dipende dalla pipeline).
executed_trades	INTEGER	YES	Numero trade eseguiti nel giorno.
closed_trades	INTEGER	YES	Numero trade chiusi nel giorno.

6.22.2 Vincoli (DuckDB)

- NOT NULL: NOT NULL
- NOT NULL: NOT NULL
- PRIMARY KEY: PRIMARY KEY(run_id, date)

6.23 005_TRACEABILITY_MATRIX.md

7 Traceability Matrix (Repo -> Docset) — v1.2.5

Build date: 2026-01-30

7.1 Scopo

Questa matrice garantisce che:

- nessun componente rilevante del repository sia “orfano” (non descritto)
- ogni sezione documentale rimandi a componenti reali
- la completezza sia verificabile tramite comandi e artefatti

7.2 0) Document set governance

Documento	Stato	Scopo
docs/000_README_DOCSET.md	CANONICO	Indice e regole del docset
docs/002_PDR_OBSERVER.md	CANONICO	Requisiti + piano implementativo
docs/009_GAP_REGISTER.md	CANONICO	Gap/backlog allineato a v1.2.5
docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md	SUPPORTO	Scenari concreti e valutazione realistica

7.3 A) Entrypoint e orchestrazione

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
app.py	Streamlit app entrypoint (routing UI pages)	IMPLEMENTATO	001§8; 003§6 (UI)	py- mstreamlitrunapp. py
main.py	CLI orchestrator (batch)	IMPLEMENTATO	003§5 (Orchestrazione)	pymain.py--help

7.4 B) Runner e utility operative (scripts/)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
scripts/sentinel.py	SENTINEL-ALPHA one-command runner.	IMPLEMENTATO	001§5-6; specs/SENTINEL_ RUNNER_SPEC; 008	pyscripts/sentinel. py--help
scripts/news_alpha.py	NEWS-ALPHA operator runner.	IMPLEMENTATO	001§5; specs/NEWS_ ALPHA_SPEC; 008	pyscripts/news_ alpha.py--help
scripts/execute.py	Execution runner (paper broker)	IMPLEMENTATO	002; 003	pyscripts/execute. py--help
scripts/ops_reset.py	Operational reset utility (NEWS-ALPHA / SENTINEL- ALPHA).	IMPLEMENTATO	001§5/Appendice A; 003§7 (Ops); 008	pyscripts/ops_reset. py--help
scripts/ops_run_session.py	Run -> pack -> certify -> pack (DB-first auto-run-id).	IMPLEMENTATO	001§5; 008 (session workflow)	pyscripts/ops_run_ session.py--help
scripts/pack_session.py	Pack reports into deterministic session folders (no symlinks).	IMPLEMENTATO	008 (packaging); 003§7	pyscripts/pack_ session.py--help
scripts/patch_persist_equity.py	Utility operativa	IMPLEMENTATO	001§5/Appendice A; 003§7 (Ops); 008	pyscripts/patch_ persist_equity.py-- help
scripts/setup.py	SENTINEL-ALPHA bootstrap / repair entrypoint (cross-platform).	IMPLEMENTATO	008 (bootstrap); 003§7	pyscripts/setup.py-- help

7.5 B2) PHASE1 DataOps (stabilità dati prezzi)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
pages/11_DataOps_Control_Room.py	Pagina Streamlit (Control Room DataOps)	IMPLEMENTATO	001§8; 003§6; 002 FR-09	(UI)
src/dataops/prices_ingest.py	DataOps: ingest incrementale prezzi + data_gaps	IMPLEMENTATO	003§4 (Data layer); 008	py-msrc.tools.dataops_prices_ingest--help
src/dataops/halts_sync.py	DataOps: sync overlay halts YAML -> DB	IMPLEMENTATO	003§4 (Data layer); 008	py-msrc.tools.dataops_sync_halts--help
src/dataops/closures_seed.py	DataOps: seed closure storiche -> market_halts	IMPLEMENTATO	003§4 (Data layer); 008	py-msrc.tools.dataops_import_closures--help
src/dataops/dq_prices.py	DataOps: DQ prezzi halt-aware (missing/stale/invalid)	IMPLEMENTATO	002 FR-09; 004	py-msrc.tools.dataops_dq_prices--help
src/tools/dataops_status.py	Tool: stato DataOps (tabella counts + quick checks)	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.dataops_status--help
src/tools/dataops_prices_ingest.py	Tool: runner ingest prezzi (offline-by-default)	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.dataops_prices_ingest--help
src/tools/dataops_sync_halts.py	Tool: runner sync halts (halts.yml)	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.dataops_sync_halts--help
src/tools/dataops_import_closures.py	Tool: seed closures CSV -> market_halts	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.dataops_import_closures--help
src/tools/dataops_dq_prices.py	Tool: runner DQ prezzi (halt-aware)	IMPLEMENTATO	002 FR-09; 004	py-msrc.tools.dataops_dq_prices--help
config/dataops/*	Config DataOps (closures seed, mapping, halts overlay)	IMPLEMENTATO	003§7 (Ops); 008	(usato dai runner)

7.6 C) Data layer (DuckDB + migrazioni)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/db/migrate.py	Schema owner + migrations DuckDB	IMPLEMENTATO	004; 003§4 (Data layer)	py-msrc.db.migrate--dbdata/sentinel_alpha.db
src/db/connection.py	Connection helper (DbConfig/connect)	IMPLEMENTATO	003§4	(usato dai runner)
src/db/audit_store.py	Persistence layer per audit artifacts	IMPLEMENTATO	003§4-5; 008	(invocato dai runner)
data/sentinel_alpha.db	DuckDB local store (schema v2.1.0)	IMPLEMENTATO	004; 008	pyscripts/sentinel.pystatus
test/test_db_migrate.py	Test: idempotenza schema + seed + tabelle execution_*	IMPLEMENTATO	004	py-mpytesttest/test_db_migrate.py

7.7 D) Execution + Risk + Monitoring

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/execution/paper_broker.py	Paper broker: ranking -> orders/fills (+ risk flags)	IMPLEMENTATO	002; 003	py-mpytesttest/test_execute_paper.py
test/test_execute_paper.py	Test: paper broker crea orders/fills e gestisce REJECTED	IMPLEMENTATO	002; 003	py-mpytesttest/test_execute_paper.py
src/risk/risk_engine.py	RiskEngine v0 (pre-trade gate)	IMPLEMENTATO	002; 003	py-mpytesttest/test_risk_engine.py
test/test_risk_engine.py	Test: RiskEngine v0 reason_code deterministico	IMPLEMENTATO	002; 003	py-mpytesttest/test_risk_engine.py
src/monitoring/tca_report.py	Monitoring/TCA v0 report (slippage/fees/cost-drag + hit-rate)	IMPLEMENTATO	002; 003	py-msrc.monitoring--help
src/monitoring/main.py	CLI Monitoring/TCA v0	IMPLEMENTATO	002; 003	py-msrc.monitoring--help
test/test_tca_report.py	Test: TCA report deterministico + alert	IMPLEMENTATO	002; 003	py-mpytesttest/test_tca_report.py

7.8 E) Core engine (src/core)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/core/audit_engine.py	Audit engine (no future leak, trades, equity)	IMPLEMENTATO	001§6; specs/AUDIT_LIFECYCLE_SPEC; 003§5	pyscripts/sentinel.pyrun--help
src/core/alert_lifecycle.py	Lifecycle monitor (TRADABLE/WAITLIST/EXPIRED + traded states)	IMPLEMENTATO	001§6; specs/AUDIT_LIFECYCLE_SPEC	py-msrc.tools.alert_lifecycle--help
src/core/cost_model.py	Retail cost model (round-trip cost)	IMPLEMENTATO	003§8 (Costi) ; 007	(usato in audit_engine)
src/core/tax_model.py	Modello fiscale IT (CGT + loss carry)	IMPLEMENTATO	003§8 (Tasse) ; 007	(usato in audit_engine)
src/core/sentiment.py	Sentiment helper (offline/deterministico)	IMPLEMENTATO	003	(usato da NEWS-ALPHA/forecast)
src/core/ticker_normalize.py	Ticker normalize helpers	IMPLEMENTATO	003	(usato da audit/forecast/tools)

7.9 F) Forecast

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/forecast/ranking.py	Forecast ranking a stelle (Wave 6)	IMPLEMENTATO	001§7; specs/WAVE6_FORECAST...; 007	py-msrc.tools.forecast_rankings--help

7.10 G) NEWS-ALPHA (src/news_alpha)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/news_alpha/run.py	News ingestion + scoring + dedup + aggregation	IMPLEMENTATO	001§5; specs/NEWS_ALPHA_SPEC; 007	pyscripts/news_alpha.pyrun--help
src/news_alpha/collect_google_news_rss.py	RSS collector (Google News)	IMPLEMENTATO	specs/NEWS_ALPHA_SPEC	(invocato da run)
src/news_alpha/db.py	Persistence layer NEWS-ALPHA	IMPLEMENTATO	003§4	(invocato da run)

7.11 H) Moduli applicativi (src/*.py)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/report_generator.py	Report generator (markdown)	IMPLEMENTATO	001; 003	py-mpytesttest/test_report_generator.py
src/performance_analyzer.py	Performance analyzer	IMPLEMENTATO	001; 003	py-mpytesttest/test_performance_analyzer.py
src/intelligence_engine.py	Intelligence engine (orchestrazione)	IMPLEMENTATO	001; 003	py-mpytesttest/test_analyst_auditor.py
src/analyst_auditor.py	Analyst/Auditor orchestrator	IMPLEMENTATO	001; 003	py-mpytesttest/test_analyst_auditor.py
src/morning_bulletin.py	Morning bulletin generator	IMPLEMENTATO	001; 003	py-mpytesttest/test_app_py_compile.py
src/sentinel_alpha.py	Application module (sentinel alpha)	IMPLEMENTATO	001; 003	py-mpytesttest/test_sentinel_core.py

7.12 I) UI Streamlit (pages/)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
pages/00__Decision__Briefing.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/01__Pipeline__Control.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/02__Gates__Data__Quality.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/03__Audit__Runs.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/04__Trades__Equity.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/05__Data__Gaps__Backfill.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/06__Forecasts__Ranking.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/07__NEWS__ALPHA.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/08__Lifecycle__Monitor.py	Pagina Streamlit	IMPLEMENTATO	001§8; 003§6	(UI)
pages/09__Execution__Log.py	Pagina Streamlit (execution_orders/execution_fills + risk flags)	IMPLEMENTATO	001§8; 003§6	(UI)
pages/10__Monitoring__TCA.py	Pagina Streamlit (TCA report)	IMPLEMENTATO	001§8; 003§6	(UI)
pages/11__DataOps__Control__Room.py	Pagina Streamlit (Control Room DataOps)	IMPLEMENTATO	001§8; 003§6; 002 FR-09	(UI)

7.13 J) GUARDIAN tooling (operativo)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
scripts/guardian.py	CLI endpoint GUARDIAN (direct mode)	IMPLEMENTATO	003§7 (Ops); 008	pyscripts/guardian.py--help
scripts/guardian__ops.py	Ops: init/sync/lint/derive/status (non-distruttivo)	IMPLEMENTATO	003§7 (Ops); 008	pyscripts/guardian.pystatus
scripts/guardian__next.py	Executor: genera CURRENT_STATE da TODO; ripresa crash	IMPLEMENTATO	003§7 (Ops); 008	pyscripts/guardian.pynext
scripts/guardian__reset.py	Utility: reset/backup GUARDIAN (ops)	IMPLEMENTATO	003§7 (Ops); 008	pyscripts/guardian__reset.py--help
.doc/TODO.md	Backlog WI (source operativa)	IMPLEMENTATO	(Ops)	pyscripts/guardian.pynext
.doc/CURRENT_STATE.md	Stato corrente + p0	IMPLEMENTATO	(Ops)	type.doc/CURRENT_STATE.md
.doc/LOGBOOK.md	Diario transizioni (audit)	IMPLEMENTATO	(Ops)	type.doc/LOGBOOK.md

7.14 K) Tooling verifiche (src/tools)

Componente (path)	Classe	Stato	Documentazione (ref)	Verifica rapida
src/tools/db_status.py	Tool: DB status / introspection	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.db_status--help
src/tools/verify_run.py	Tool: verify run artifacts	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.verify_run--help
src/tools/verify_inputs.py	Tool: verify inputs	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.verify_inputs--help
src/tools/verify_ticker_mappings.py	Tool: verify ticker mappings	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.verify_ticker_mappings--help
src/tools/verify_provenance.py	Tool: verify provenance	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.verify_provenance--help
src/tools/forecast_rankings.py	Tool: forecast rankings	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.forecast_rankings--help
src/tools/forced_exits.py	Tool: forced exits analysis	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.forced_exits--help
src/tools/ticker_mappings.py	Tool: ticker mappings ops	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.ticker_mappings--help
src/tools/universe_membership.py	Tool: universe membership ops	IMPLEMENTATO	003§7 (Ops); 008	py-msrc.tools.universe_membership--help

7.15 Regola di accettazione (operativa)

- **Coverage 100% sui componenti rilevanti:** runner, pipeline, data layer, UI, strumenti, tooling operativo.
- **Zero voci fantasma:** nulla è descritto come “presente” se non esiste nello snapshot.
- **Verifiche ripetibili:** per ogni runner esiste almeno un comando --help o status che conferma l’installazione.

7.16 006_REPO_BOM.md

7.17 doc_id: 006_REPO_BOM docset_version: 1.2.5 status: canonical
last_updated: 2026-01-26

8 Repository BOM (Bill of Materials) — v1.2.5

Build date: 2026-01-26

8.1 1. Scopo

Inventario dei componenti *as-built* nello snapshot repository, con focus su ciò che è rilevante per:

- pipeline operativa (runner CLI)
- UI Streamlit
- moduli core (audit/forecast/news)
- risk & execution (paper-first)
- governance docset (GUARDIAN)

8.2 2. Struttura ad alto livello

- app.py — entrypoint Streamlit (UI)
- main.py — orchestrazione CLI/batch
- scripts/ — runner e utility operative (sentinel/news/execute/ops/guardian)
- src/ — logica applicativa

- data/ — DuckDB locale e file di supporto
- reports/ — artefatti e session logs (per riproducibilità)
- .doc/ — libreria documentale derivata (GUARDIAN)
- docs/ — documentazione canonica e PDF

8.3 3. Runner e utility (scripts/)

File	Ruolo
scripts/sentinel.py	Runner principale (migrate/test/run/certify/verify/forecast/status)
scripts/news_alpha.py	Runner NEWS-ALPHA (collect/run/status)
scripts/execute.py	Execution runner (paper-first)
scripts/ops_run_session.py	Workflow run→pack→certify→pack
scripts/pack_session.py	Packaging deterministico sessione
scripts/ops_reset.py	Reset operativo (news/sentinel)
scripts/setup.py	Bootstrap/repair cross-platform
scripts/guardian.py	Governance docset (sync/lint/derive/status)

8.4 4. UI Streamlit (pages/)

File	Ruolo
pages/00_Decision_Briefing.py	Decision briefing / sintesi
pages/01_Pipeline_Control.py	Controllo pipeline
pages/02_Gates_Data_Quality.py	Gates / data quality
pages/03_Audit_Runs.py	Audit runs
pages/04_Trades_Equity.py	Trades & equity
pages/05_Data_Gaps_Backfill.py	Data gaps / backfill
pages/06_Forecasts_Ranking.py	Forecasts / ranking
pages/07_NEWS_ALPHA.py	NEWS-ALPHA console
pages/08_Lifecycle_Monitor.py	Lifecycle / monitor

8.5 5. Package applicativi (src/)

Package	Contenuto
src/core/	audit engine, cost model, tax model, ticker normalization
src/db/	schema owner + connection + audit store
src/news_alpha/	collect, deterministic sentiment, persistence
src/forecast/	stars/ranking (Wave 6)
src/risk/	risk gate (baseline)
src/execution/	paper broker baseline
src/monitoring/	monitor/metrics (baseline)
src/tools/	verify utilities e diagnostica
src/data/	helper per data layer

8.6 6. Note di release “pulita”

Per packaging distributivo si raccomanda di escludere:

- __pycache__/, *.pyc
- reports/ storici non necessari (includere solo evidence pack selezionato)
- .old/ e snapshot storici

8.7 007_PARAMETER_SNAPSHOT.md

8.8 doc_id: 007_PARAMETER_SNAPSHOT docset_version: 1.2.5 status: canonical last_updated: 2026-01-26

9 Parameter Snapshot (estratto dal codice) — v1.2.5

Build date: 2026-01-26

Scopo: ridurre divergenze doc-vs-code riportando i **default** e le costanti “contract-like”.

9.1 1. Cost model (retail realism)

- `src/core/cost_model.py`
 - `CostModel.round_trip_cost_pct=0.0075` (0.75% round-trip, split entry/exit)

9.2 2. Tax model (Italia, semplificato)

- `src/core/tax_model.py`
 - `ItalianTaxModel.capital_gains_rate=0.26`
 - `loss_carry` (zainetto fiscale) gestito in simulazione

9.3 3. Risk gate (baseline)

- `src/risk/risk_engine.py`
 - `RiskConfig.max_positions=10`
 - `RiskConfig.cash_reserve_pct=0.20`
 - `RiskConfig.max_position_pct=0.20`
 - `RiskConfig.risk_scalar=1.0`

9.4 4. Forecast ranking (Wave 6)

- `src/forecast/ranking.py` (Spec: WAVE6_FORECAST_STARS_RANKING_SPEC.mdv0.1)
 - `N_MIN=20`
 - `N_CONF=60`
 - `W_GLOBAL=0.25`
 - `K_SENT=0.20`
 - `DEFAULT_TOP_N=25`
 - `DEFAULT_EXCLUDE_EXIT_REASON="FALLBACK_LAST_PRICE"`

9.5 5. NEWS-ALPHA deterministic sentiment

- `src/news_alpha/sentiment.py`
 - `MODEL="lexicon-v1"`
 - `scoring: (pos-neg)/(pos+neg)` in `[-1, +1]` su token `[A-Za-z]+`

9.6 6. Execution (paper)

- `src/execution/paper_broker.py`
 - `starting_cash` default: 100000.0
 - `order type: MARKET (paper)`, fill immediato, fees via `CostModel.entry_cost()`

9.7 008_EVIDENCE_PACK.md

9.8 doc_id: 008_EVIDENCE_PACK docset_version: 1.2.5 status: canonical last_updated: 2026-01-26

10 Evidence Pack — v1.2.5

Build date: 2026-01-26

Questo documento elenca comandi “verificabili” per dimostrare che lo snapshot è eseguibile e coerente.

Nota: su Windows/PowerShell usare `py` invece di `python`.

10.1 1. Sanity checks

- Test suite: `py-mpytest`
- Stato GUARDIAN: `pyscripts/guardian.pystatus`
- Lint docset: `pyscripts/guardian.pylint`

10.2 2. Data layer (DuckDB)

- Migrazione schema: `py-msrc.db.migrate--dbdata/sentinel_alpha.db`

10.3 3. Pipeline sentinel (audit/backtest)

- Help runner: `pyscripts/sentinel.py--help`
- Status: `pyscripts/sentinel.pystatus`
- Certify (offline default): `pyscripts/sentinel.pycertify`

Artefatti attesi:

- record in `audit_runs`, `audit_trades`, `audit_equity`
- `transcript/summary` in `reports/`

10.4 4. NEWS-ALPHA

- Help runner: `pyscripts/news_alpha.py--help`
- Status: `pyscripts/news_alpha.pystatus`
- Run (offline, se configurato con fixtures): `pyscripts/news_alpha.pyrun`

Artefatti attesi:

- `sentiment_cache` popolata
- `report/JSONL` in `reports/news_alpha/` (se abilitato)

10.5 5. Forecast ranking (Wave 6)

- Help: `py-msrc.forecast.ranking--help` (se esposto) oppure via `sentinel`
- Generate ranking: `pyscripts/sentinel.pyforecast--universeALL--top-n25`

Artefatti attesi:

- output ranking deterministico + stars
- record/report associati a `run_id`

10.6 6. Execution (paper-first)

- Help: `pyscripts/execute.py--help`
- Paper execution: `pyscripts/execute.py--top-n10--starting-cash100000`

Artefatti attesi:

- tabelle `execution_orders` e `execution_fills` aggiornate
- report execution (se presente) e tracciabilità via `run_id`

10.7 7. Packaging sessione

- `pyscripts/pack_session.py--help`
- `pyscripts/ops_run_session.py--help`

10.8 8. Docset governance

- Sync canonici: `pyscripts/guardian.pysync--clean`
- Derive brief: `pyscripts/guardian.pyderive`

10.9 009_GAP_REGISTER.md

10.10 doc_id: 009_GAP_REGISTER docset_version: 1.2.5 status: canonical
last_updated: 2026-01-26

10.11 Indice gap (ID stabili)

This index provides **stable identifiers** used by:

- docs/010_MODULE_REGISTRY.md (Derived gaps column)
- docs/011_GAP_DERIVATION_MATRIX.md (root gaps)
- MkDocs navigation

Gap ID	Area	Title
GAP-OMS	Execution	Order Management System (OMS)
GAP-BROKER-ADAPTER	Execution	Live broker adapter + kill-switch
GAP-EXEC-LOG	Execution/Monitoring	Stable execution log (orders/fills lifecycle)
GAP-RISK-HARDEN	Risk	Hardened Risk Engine (dynamic stops, limits, kill-switch policy)
GAP-GUARDRAILS	Risk/Ops	Behavioral guardrails (discipline mode, cooling-off)
GAP-ALERTING	Monitoring/Ops	Alerting & notification channels
GAP-DRIFT	Monitoring	Drift detection + model monitoring
GAP-CA-COLLECTOR	Data	Corporate actions collector (splits/dividends/events)
GAP-MACRO-LAYER	Data/Signal	Macro data layer + regime detection
GAP-PAIRS-ENGINE	Signal/Execution	Pairs trading engine (selection, z-score, neutrality)

10.12 Dettagli gap (per ID)

10.12.1 GAP-OMS — Order Management System (OMS)

- **Driver:** missing order state machine (NEW→SUBMITTED→FILLED/CANCELLED/REJECTED) + replay-safe reconciliation.
- **Blocks:** broker adapter, trade ticket, reliable alerts, post-trade attribution.
- **Acceptance (minima):** persistent state; deterministic IDs; validated transitions; replay-safe.

10.12.2 GAP-BROKER-ADAPTER — Live broker adapter + kill-switch

- **Driver:** no real broker interface + no hard kill-switch boundary.
- **Blocks:** LIVE-GRADE; real fills; real-time exposure control.
- **Acceptance (minima):** adapter interface; sandbox; emergency stop; full audit.

10.12.3 GAP-EXEC-LOG — Stable execution log

- **Driver:** orders/fills logging not yet a stable contract for monitoring/analytics.
- **Blocks:** TCA, slippage calibration, attribution, drift baselines.
- **Acceptance (minima):** immutable fills; consistent order↔fill↔portfolio link; latency/fees captured.

10.12.4 GAP-RISK-HARDEN — Hardened Risk Engine

- **Driver:** current checks are basic; not sufficient for safe paper/live.
- **Blocks:** guardrails, dynamic stops, concentration, kill-switch policy.
- **Acceptance (minima):** pre+post trade; stop framework; global limits; tested edge cases.

10.12.5 GAP-GUARDRAILS — Behavioral guardrails

- **Driver:** no discipline-mode / cooling-off / override policy.
- **Blocks:** safe retail ops; reduces user-error failure mode.
- **Acceptance (minima):** cooling-off after stop; max overrides; hard blocks.

10.12.6 GAP-ALERTING — Alerting & notifications

- **Driver:** no alert lifecycle (create→route→ack→close).
- **Blocks:** sustainable ops; incident response; user trust.
- **Acceptance (minima):** rules; channels (local + email/webhook); ack logging.

10.12.7 GAP-DRIFT — Drift detection + model monitoring

- **Driver:** missing KPIs/thresholds and baselines to detect drift/decay.
- **Blocks:** safe signal evolution; early warning for alpha decay.
- **Acceptance (minima):** baselines; thresholds; periodic evaluation; alert integration.

10.12.8 GAP-CA-COLLECTOR — Corporate actions collector

- **Driver:** no automated CA ingestion/normalization; timing errors likely.
- **Blocks:** event-driven strategies; correct price adjustments.
- **Acceptance (minima):** collector; mapping; timing rules; CA audit entries.

10.12.9 GAP-MACRO-LAYER — Macro data layer + regime detection

- **Driver:** missing macro inputs and regime classifier.
- **Blocks:** tactical rotation; regime-aware weighting.
- **Acceptance (minima):** macro ingest; features; regime labels; backfill+validation.

10.12.10 GAP-PAIRS-ENGINE — Pairs trading engine

- **Driver:** missing pair selection, spread monitoring, neutrality enforcement.
- **Blocks:** scenario 5.
- **Acceptance (minima):** selection; z-score pipeline; breakdown detection; execution hooks.

11 GAP Register — v1.2.5 (as-built vs target)

Build date: 2026-01-26

11.1 1. Scopo

Registrare in modo auditabile:

- cosa è già implementato (as-built)
- cosa è parziale/basilare
- cosa manca per conformità v1.2.5 e per scenari operativi

Riferimento scenari: docs/use_cases/SCENARI_APPLICATIVI_v1.2.5.md

11.2 2. Priorità e codifica

- **P0 (bloccante):** impedisce paper robusto / qualsiasi live readiness
- **P1 (alta):** necessario per sostenibilità e qualità retail
- **P2 (media):** differenziazione / alpha expansion
- **P3 (nice-to-have)**

11.3 3. Gap principali per layer

11.3.1 Layer 1 — Risk & Execution (P0/P1)

Gap	Stato as-built	Impatto	Target (milestone)
RiskEngine evoluto (stop/trailing/kill-switch/cooldown)	PARZIALE (gate statico)	P0	M1
OMS lifecycle + posizione/PNL accounting	ASSENTE (solo orders/fills)	P0	M1
Slippage/spread model realistico + TCA	ASSENTE	P1	M1
Reconciliation + idempotenza ordini/fills	ASSENTE	P1	M1
BrokerAdapter interface (paper/live)	PARZIALE (paper only)	P1	M1→M2

11.3.2 Layer 2 — Alpha enhancement (P2)

Gap	Scenario	Stato as-built	Target
Factor library estesa (value/quality)	2	ASSENTE	M2
Dynamic weighting regime-aware	2,4	ASSENTE	M2/M4
Corporate actions collector + event engine	3	ASSENTE	M3
Macro data layer + regime detection	4	ASSENTE	M4
Pairs selection/monitoring + market-neutral exec	5	ASSENTE	M4

11.3.3 Layer 3 — Feedback & Monitoring (P1)

Gap	Stato as-built	Target
Model monitoring rolling + alert thresholds	BASICO	M1/M2
Drift detection (features/target)	ASSENTE	M2/M3
Alert center (UI + notifiche)	BASICO	M1

11.3.4 Layer 4 — UI/UX operativa (P2)

Gap	Stato as-built	Target
Trade ticket / execute workflow guidato	BASICO	M1
What-if analysis (risk preview)	ASSENTE	M1/M2
One-click execution con conferme e guardrails	ASSENTE	M1

11.4 4. Note di realismo

- Le stime FTE sono sensibili a: broker scelto, qualità fonti dati (corporate actions/macro), e profondità monitoring richiesta.
- Il “paper trading funzionante” è più vicino (perché paper broker e gate esistono), ma il salto a *paper robusto* richiede OMS + post-trade + monitoring.

12 Technical Specs (docs/specs)

12.1 specs/AUDIT_LIFECYCLE_SPEC.md

13 AUDIT_LIFECYCLE_SPEC.md

Spec version: v0.1

Build date: 2026-01-24

13.1 Scopo

Formalizzare gli stati lifecycle e il decision ledger, in modo che:

- la UI possa rappresentare correttamente stati e transizioni
- i motivi di esclusione siano espliciti (auditability)
- le “eccezioni” (fallback, halt, right-censor) siano disclosure-first

13.2 Stati (concettuali)

13.2.1 Stati pre-trade (eligibility)

- **CANDIDATE**: segnale presente in recs per la data as-of.
- **ELIGIBLE**: passa normalize + ticker_mappings + universe_membership.
- **ENTERABLE**: esiste intended_buy_date (prezzo disponibile dopo il segnale).
- **WAITLIST**: temporaneamente non enterable per halt o gap dati (policy-dependent).
- **DROPPED**: scartato (right-censored, missing prices, fuori universo, dedup).

13.2.2 Stati trade (execution)

- **EXECUTED**: trade creato in audit_trades.
- **CLOSED**: trade chiuso con sell_date e exit_reason.
- **FALLBACK_EXIT**: uscita con prezzo fallback (disclosure).

13.3 Decision ledger

Tabella: audit_signal_decisions

Minimi campi richiesti:

- ticker_original, ticker, firm, rating, universe_id, signal_date
- intended_buy_date vs buy_date (+ exec_shift_sessions)
- intended_sell_date vs sell_date (+ exit_shift_sessions)
- decision (EXECUTED/SKIPPED/DROPPED_DEDUP)
- skip_reason e/o halt_reason

13.4 Regole di disclosure

- Ogni skip deve avere motivazione classificabile (skip_reason/reason_code).
 - Ogni fallback deve essere marcato come tale e idealmente escluso dalla calibrazione forecast (default exclude_exit_reason).
-

13.5 specs/NEWS_ALPHA_SPEC.md

14 NEWS_ALPHA_SPEC.md

Spec version: v0.1

Build date: 2026-01-24

14.1 Scopo

Definire la lane NEWS-ALPHA:

- raccolta (RSS/History) con posture online guardata

- scoring sentiment deterministico
- dedup e aggregazione
- persistenza in DuckDB (recs + sentiment_cache)

Runner: scripts/news_alpha.py

14.2 Postura online (guardrail)

Azioni online richiedono **ENTRAMBI**:

- --online
- NEWS_ALPHA_ALLOW_ONLINE=1 oppure --allow-online

Motivazione: evitare cambi non deterministici e rispettare l'approccio offline-by-default.

14.3 Pipeline RSS (collect)

Input:

- intervallo date (date-from, date-to)
- query window when-days
- eventuale allowlist domini

Output:

- raw RSS XML (se online) in reports/news_alpha/raw/rss
- fixtures JSONL in reports/news_alpha/collector/collector_<ts>.jsonl
- stats JSON con conteggi e qualità

Gate opzionale:

- --strict-dq: fallisce se zero items “kept”.

14.4 Pipeline run (fixtures -> DuckDB)

Input:

- fixtures JSONL
- intervallo date

Azioni:

- scoring + dedup
- mapping a ticker
- calcolo rating discreto da score medio:
 - BUY se score ≥ 0.20
 - DOWNGRADE se score ≤ -0.20
 - HOLD altrimenti

Output:

- write in recs (firm lane news-alpha) e sentiment_cache
- rejects JSONL opzionale con motivazione
- log file opzionale (plain o JSONL)

14.5 History lane (GDELT)

Il runner include comandi history (download/profile/fixtures) per alimentare dataset storici, sempre con guardrail online.

14.6 specs/SENTINEL_RUNNER_SPEC.md

15 SENTINEL_RUNNER_SPEC.md

Spec version: v0.1

Build date: 2026-01-24

15.1 Scopo

Definire il contratto operativo del runner scripts/sentinel.py:

- CLI minimale e stabile
- run_id, transcript e posture offline/online
- integrazione con schema owner e tool di verifica

15.2 Comandi supportati

- migrate: crea/aggiorna schema DuckDB e seed universi base
- test: esegue suite test
- run: esegue audit run (operativo)
- certify: esegue audit run in postura “certification-grade” (offline-by-default)
- verify: verifica un run_id (consistenza e policy)
- forecast: genera ranking a stelle (Wave 6)
- status: summary DB e ambiente

15.3 Run ID

- Variabile ambiente: SENTINEL_RUN_ID
- Se assente e il comando e' in {"run","certify","verify"}, il runner genera un run_id e lo preserva.

15.4 Transcript

- run -> reports/RUN_TRANSCRIPT_<run_id>.txt
- certify -> reports/CERTIFY_TRANSCRIPT_<run_id>.txt
- Transcripts sono best-effort: non bloccano l'esecuzione.

15.5 Offline/Online posture

- Flag:
 - --offline forza offline
 - --online abilita online backfill
- Precedenza:
 - 1) --online
 - 2) --offline / --no-backfill
 - 3) certify default offline

Variabili ambiente rilevanti:

- SENTINEL_ALLOW_ONLINE_BACKFILL
- SENTINEL_OFFLINE
- SENTINEL_PRICE_PROVIDER_ORDER
- SENTINEL_DISABLE_YFINANCE (default 1)

15.6 Disclosure defaults (retail)

Il runner imposta:

- SENTINEL_DIVIDEND_POLICY=B
- SENTINEL_TIMING_MODE=T_PLUS_1

Nota: nello snapshot questi switch sono *disclosure* (registrati nei report) e non necessariamente alterano tutta la logica economica.

15.7 specs/WAVE6_FORECAST_STARS_RANKING_SPEC.md

16 WAVE6_FORECAST_STARS_RANKING_SPEC.md

Spec version: v0.1

Build date: 2026-01-24

16.1 Scopo

Definire una procedura deterministica per produrre:

- forecast_return_pct (proxy rendimento atteso)
- confidence (affidabilità della stima)
- stars (1..5) come sintesi retail
- ranking deterministico (tie-break stabili)

Questa spec è implementata in src/forecast/ranking.py.

16.2 Input

- Tabelle DuckDB:
 - recs (segnali con firm, rating, sentiment_score, headline, url)
 - audit_trades (storico trade per calibrazione)
 - prices (per determinare enterability / buy_date)
 - universe_membership, ticker_mappings (eligibility)
- Parametri (dal codice):
 - N_MIN = 20
 - N_CONF = 60
 - W_GLOBAL = 0.25
 - K_SENT = 0.20
 - DEFAULT_TOP_N = 25
 - DEFAULT_EXCLUDE_EXIT_REASON = FALLBACK_LAST_PRICE

16.3 Output

Un dict JSON-serializzabile con:

- metadata: spec_name, spec_version, asof_date, universe_id, top_n
- diagnostics: candidates_total, enterable_total, dropped_missing_prices, dropped_right_censored, calibration_global_n, calibration_global_mean_return_pct
- rows: lista segnali con forecast/confidence/stars e breakdown
- by_firm: aggregazioni per firm

16.4 Algoritmo (passi)

16.4.1 Step 0 - Determine as-of date

- asof_date = max(recs.date) nel perimetro universe_id (ALL o filtro).

16.4.2 Step 1 - Load candidate signals

- filtrare recs in data = asof_date
- normalizzare ticker (normalize_ticker_sql)
- applicare ticker_mappings time-bounded
- join con universe_membership time-bounded (survivorship control)
- calcolare intended_buy_date=MIN(prices.date)>signal_date
- segnali senza intended_buy_date sono “non enterable” (right-censored o missing).

16.4.3 Step 2 - Load calibration stats (audit_trades)

- usare trade storici con signal_date<asof_date
- escludere (default) exit_reason=DEFAULT_EXCLUDE_EXIT_REASON per evitare contaminazione da fallback

- calcolare:
 - bucket_stats per (firm, rating): n, mean_return_pct, stdev_return_pct, last_signal_date
 - firm_stats per firm: n, mean_return_pct
 - global_stats: n, mean_return_pct

16.4.4 Step 3 - Shrinkage (robustezza piccoli campioni)

Per ogni bucket (firm, rating):

- se $n_bucket \geq N_MIN$: $shrunk = bucket_mean$
- altrimenti:
 - $\alpha = n_bucket / N_MIN$
 - blend verso firm_mean: $bucket_mean\alpha + firm_mean(1-\alpha)$
 - blend verso global_mean con prior_weight = $\max(W_GLOBAL, 1-\alpha)$

16.4.5 Step 4 - Sentiment adjustment

- clamp sentiment_score in $[-1, +1]$
- $sent_adj = sentiment_score * K_SENT$

16.4.6 Step 5 - Forecast e confidence

- $forecast_return_pct = shrunk_return_pct + sent_adj$
- $confidence = \min(1, n_bucket / N_CONF)$

Fallback: se $global_n \leq 0$ (nessun trade storico)

- $shrunk_return_pct = 0$
- $confidence = 0$
- $sent_adj = sentiment_score * 0.50$
- $forecast = sent_adj$

16.4.7 Step 6 - Ranking e stars

- ordinare per:
 - 1) forecast_return_pct desc
 - 2) confidence desc
 - 3) rating asc (tie-break deterministico)
 - 4) firm asc
 - 5) ticker_effective asc
- assegnare stars per percentile:
 - 5 stelle: top 10%
 - 4 stelle: successivo 20%
 - 3 stelle: successivo 40%
 - 2 stelle: successivo 20%
 - 1 stella: bottom 10%

16.5 Disclosure / retail notes

- Forecast non e' una "promessa": e' una metrica comparativa basata su storico e sentiment.
- Confidence e' esplicita per evitare falsa precisione su bucket piccoli.